

# KMG Security Agent

## Архитектура и механизм полного анализа

Решение для хакатона KMG Digital. Агент проверяет исходники, конфигурацию и документацию Git-проекта по ИБ-01–ИБ-08 и технической спецификации. Использует DeepSeek API; модель запуска фиксируется в отчёте. Основная проверка выполняется без участия человека и без изменения тестового проекта.

### 1. Интеграция и снимок

GitHub Actions запускает CLI отдельным заданием после push либо ручного dispatch. Доверенный агент и проверяемый target размещаются в разных каталогах. Версия агента закрепляется SHA. Проверяются чистота target, его commit и контрольные суммы файлов. Артефакты записываются вне target; в конце снимок проверяется повторно.

### 2. Сбор контекста

Учитываются все отслеживаемые файлы, а не только git diff. Читаются исходники, зависимости, CI-конфигурация и DOCX. Для Python извлекаются AST-факты. Исключения бинарных ресурсов и потенциально секретных файлов отражаются в инвентаризации. Неизвестный формат не пропускается молча: аудит завершается ошибкой.

### 3. Поэтапный анализ

Тексты делятся на фрагменты. На этапе map DeepSeek возвращает наблюдения по каждому фрагменту. Далее review сопоставляет наблюдения и исходники с каждым требованием; модель может запросить дополнительные диапазоны файлов. Небольшие общие компоненты включаются в контекст, чтобы учитывать middleware, маршруты, настройки и аудит.

### 4. Подтверждение выводов

Кандидаты проходят verify с обращением к исходным доказательствам. Верификатор может запросить ещё два раунда исходников. Основные доказательства сохраняются целиком; подбор дополнительного контекста ограничен бюджетом и явно обозначен. JSON проверяется схемой; пути и диапазоны строк должны существовать. Для неопределённого требования предусмотрены ограниченные дополнительные разборы. Неподтверждённый кандидат не превращается автоматически в нарушение или успешный статус.

# Результаты и ограничения

## Отчёт и решение CI

JSON и Markdown включают SHA, время, длительность, статусы всех восьми требований, находки, фрагменты кода, критичность и рекомендации. ИБ-нарушения блокируют pipeline. Дефекты EXTRA сохраняются отдельно и сами по себе не блокируют доставку. Код 0 означает завершение без обязательных нарушений, 1 — завершение с нарушениями, 2 — неполную проверку или ошибку.

## Полнота и воспроизводимость

Счётчик обработанных фрагментов и полный перечень файлов позволяют проверить охват входных материалов. Это не математическая гарантия полноты выводов LLM. Кэш промежуточных наблюдений привязан к содержимому, модели и инструкциям; итоговые проверки выполняются заново. Итоговый официальный прогон использует --no-cache и неизменённый commit организаторов.

## Ошибки, время и ресурсы

CLI ограничен deadline, GitHub job — 30 минутами. Родительский процесс завершает зависший worker. При таймауте сохраняется частичный отчёт; при ошибке внешнего провайдера — диагностика без отчёта. Локальный бюджет ограничивает токены и попытки запросов, включая повторы; его исчерпание сейчас сохраняет incomplete. Эта трактовка отдельно указана в ограничениях комплекта. Фактическое usage и длительность приложены к результатам запуска.

## Защита процесса

Исходники и текст модели считаются недоверенными данными. Агент не исполняет код target в задании анализа; ключ находится в GitHub Secrets или внешнем .env. Перед API и записью результатов применяется эвристическое маскирование секретов. В демонстрационном helpdesk-demo main защищён обязательным Security merge gate: ошибки, отменённые и пропущенные задания не дают допуск.

## Границы решения

Статический анализ не подтверждает реальные ACL, инфраструктуру и сертификацию. Модель может пропускать ошибки и давать ложные срабатывания; verify использует тот же провайдер. Закрывающая эталонная разметка отсутствует. UI и предложения патчей — дополнительные функции; исправления не применяются к официальному target. Слияние разрешённого исправления остаётся решением разработчика.